IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Corrections détaillées

Jour 3 — Probabilités et classification

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Exercice 1

★☆☆ Fondamental

Rappel : $P(A) = \frac{cas\ favorables}{total}$.

Le mini-dataset contient 8 développeurs : A, B, C, D, E, F, G, H. Les développeurs avec salaire_eleve = 1 sont : C, D, F, H (4 sur 8).

a) $P(\text{salaire_eleve} = 1) = \frac{4}{8} = \boxed{0{,}5}$

b) $P(\text{salaire_eleve} = 0) = \frac{4}{8} = \boxed{0{,}5}$

Vérification : $P(0) + P(1) = 0{,}5 + 0{,}5 = 1$. ✓

c) Développeurs avec expérience ≥ 6 : C (6), D (8), F (10), H (7) — 4 sur 8. $P(\text{exp} \geq 6) = \frac{4}{8} = \boxed{0{,}5}$

d) Développeurs avec exp ≥ 6 **et** salaire_eleve = 1 : C, D, F, H — les 4 mêmes. Réponse : $\boxed{4}$ développeurs.

⚠ **Erreurs fréquentes**

- Confondre $P(A)$ et le **nombre** de cas favorables. $P(A)$ est une **proportion**, pas un compte.
- Oublier que $P(A) + P(\bar{A}) = 1$ toujours.

Exercice 2

★☆☆ Fondamental

Rappel : $P(A \mid B) = \frac{nb\ avec\ A\ et\ B}{nb\ avec\ B}$.

a) Développeurs avec exp ≥ 6 : C, D, F, H (4 dev). Parmi eux, salaire_eleve = 1 : C, D, F, H (4 dev).

$$P(\text{sal_eleve} = 1 \mid \text{exp} \geq 6) = \frac{4}{4} = \boxed{1{,}00}$$

b) Développeurs avec exp < 6 : A (2), B (4), E (3), G (1) (4 dev). Parmi eux, salaire_eleve = 1 : aucun (tous ont un salaire ≤ 45 k€).

$$P(\text{sal_eleve} = 1 \mid \text{exp} < 6) = \frac{0}{4} = \boxed{0{,}00}$$

c) Avoir ≥ 6 ans d'expérience change **radicalement** la probabilité : elle passe de 0% à 100%. L'expérience est un très bon prédicteur sur ce mini-dataset.

d) Ratio : $\frac{1{,}00}{0{,}00}$ — techniquement infini car le dénominateur est 0. En pratique, on dirait que l'expérience ≥ 6 "garantit" un salaire élevé (sur ce petit échantillon).

À retenir

La probabilité conditionnelle permet de quantifier à quel point une feature est **informative** pour prédire la classe. C'est la base de nombreux classifieurs.

Exercice 3

★☆☆ **Fondamental**

$g(\mathbf{x}) = 0{,}5\,x_1 + 0{,}8\,x_2 - 4.$

---

a) Calculs :

Dev	x_1	x_2	$g(\mathbf{x}) = 0{,}5x_1 + 0{,}8x_2 - 4$	Signe	$\hat{y}$	y
1	2	3	$1{,}0 + 2{,}4 - 4 = \mathbf{-0{,}6}$	$-$	$\boxed{0}$	0
2	10	6	$5{,}0 + 4{,}8 - 4 = \mathbf{+5{,}8}$	$+$	$\boxed{1}$	1
3	5	4	$2{,}5 + 3{,}2 - 4 = \mathbf{+1{,}7}$	$+$	$\boxed{1}$	0

b) Dev 1 : correct (✓). Dev 2 : correct (✓). Dev 3 : **incorrect** (×).
Accuracy : $\frac{2}{3} = \boxed{66{,}7\%}$.
c) Le dev 3 est prédit 1 alors que le vrai label est 0 : c'est un **faux positif** (*false positive*).

Erreurs fréquentes

- Oublier le biais -4 dans le calcul de $g(\mathbf{x})$.
- Confondre faux positif (prédit 1, vrai 0) et faux négatif (prédit 0, vrai 1).

Exercice 4

★☆☆ **Fondamental**

$g(\mathbf{x}) = 0{,}5\,x_1 + 0{,}8\,x_2 - 4 = 0.$

---

a) Frontière $g(\mathbf{x}) = 0$:

$$0{,}5\,x_1 + 0{,}8\,x_2 - 4 = 0 \implies x_2 = -\frac{0{,}5}{0{,}8}\,x_1 + \frac{4}{0{,}8} = \boxed{-0{,}625\,x_1 + 5}$$

b) Deux points :

- $x_1 = 0$: $x_2 = 5{,}0$ — point $(0;\ 5)$
- $x_1 = 8$: $x_2 = -0{,}625 \times 8 + 5 = 0{,}0$ — point $(8;\ 0)$

c) La droite passe par $(0, 5)$ et $(8, 0)$. Au-dessus de la droite ($g > 0$) : zone "classe 1". En-dessous ($g < 0$) : zone "classe 0".

d) Le dev 3 ($x_1 = 5, x_2 = 4$) est **au-dessus** de la droite ($g = +1{,}7 > 0$), donc dans la zone "classe 1". Mais son vrai label est 0 : il est du **mauvais côté**.

À retenir

La frontière $\mathbf{w}^T\mathbf{x} + b = 0$ est une **droite** en 2D. D'un côté : classe 0, de l'autre : classe 1. Les erreurs sont les points du mauvais côté.

Exercice 5

★★☆ Intermédiaire

$\alpha = 1$, $\mathbf{w} = (0,0)^T$, $b = 0$. *Mise à jour :* $\mathbf{w} + \alpha(y - \hat{y})\mathbf{x}$.

Itération $i = 1$ — $\mathbf{x}_1 = (3,4)^T$, $y_1 = 1$:
$g = (0)(3) + (0)(4) + 0 = 0 \leq 0$, donc $\hat{y}_1 = 0$. Erreur $(0 \neq 1)$.
$\mathbf{w}_{\text{new}} = (0,0) + 1 \times (1-0) \times (3,4) = \boxed{(3,4)}$, $b_{\text{new}} = 0 + 1 \times 1 = \boxed{1}$.

Itération $i = 2$ — $\mathbf{x}_2 = (1,1)^T$, $y_2 = 0$:
$g = (3)(1) + (4)(1) + 1 = 8 > 0$, donc $\hat{y}_2 = 1$. Erreur $(1 \neq 0)$.
$\mathbf{w}_{\text{new}} = (3,4) + 1 \times (0-1) \times (1,1) = (3-1, 4-1) = \boxed{(2,3)}$, $b_{\text{new}} = 1 + 1 \times (-1) = \boxed{0}$.

Itération $i = 3$ — $\mathbf{x}_3 = (5,2)^T$, $y_3 = 1$:
$g = (2)(5) + (3)(2) + 0 = 16 > 0$, donc $\hat{y}_3 = 1$. Pas d'erreur $(1 = 1)$.
Pas de mise à jour. $\mathbf{w} = (2,3)$, $b = 0$ inchangés.

Tableau complet :

i	$\mathbf{w}$ avant	b	$g(\mathbf{x}_i)$	$\hat{y}_i$	Erreur ?	$\mathbf{w}$ après
1	$(0,0)$	0	0	0	Oui	$(3,4)$, b=1
2	$(3,4)$	1	8	1	Oui	$(2,3)$, b=0
3	$(2,3)$	0	16	1	Non	$(2,3)$, b=0

Erreurs fréquentes

- Oublier de mettre à jour le **biais** b en plus des poids $\mathbf{w}$.
- Se tromper dans le signe de $(y_i - \hat{y}_i)$: si $y = 0$ et $\hat{y} = 1$, le facteur est -1.
- Mettre à jour alors qu'il n'y a **pas d'erreur** — le perceptron ne change rien quand $\hat{y} = y$.

Exercice 6

★★☆ Intermédiaire

a-b) Les 4 points : $(0,0)$ rouge, $(0,1)$ vert, $(1,0)$ vert, $(1,1)$ rouge. Les rouges sont en diagonale, les verts aussi. Aucune droite ne peut les séparer.

c) Démonstration par l'absurde. Supposons qu'il existe w_1, w_2, b tels que $g(\mathbf{x}) = w_1x_1 + w_2x_2 + b$ classifie correctement :

$$g(0,0) = b \leq 0 \qquad \text{(classe 0)} \qquad \text{(I)}$$
$$g(0,1) = w_2 + b > 0 \qquad \text{(classe 1)} \qquad \text{(II)}$$
$$g(1,0) = w_1 + b > 0 \qquad \text{(classe 1)} \qquad \text{(III)}$$
$$g(1,1) = w_1 + w_2 + b \leq 0 \qquad \text{(classe 0)} \qquad \text{(IV)}$$

De (II) : $w_2 > -b$. De (III) : $w_1 > -b$.

Donc $w_1 + w_2 > -2b$. De (I) : $b \leq 0$, donc $-2b \geq 0$, donc $w_1 + w_2 > 0$.

Mais (IV) exige $w_1 + w_2 + b \leq 0$, i.e. $w_1 + w_2 \leq -b$. De (I) : $-b \geq 0$, mais on a montré $w_1 + w_2 > 0$ tandis que (IV) + (II) + (III) donnent : $(w_1 + b) + (w_2 + b) > 0$ mais $w_1 + w_2 + b \leq 0$, soit $(w_1 + b) + (w_2 + b) - b \leq 0$. Avec $b \leq 0$: contradiction. □

d) Le problème n'est pas linéairement séparable car aucun hyperplan ne peut séparer les classes. Il faut un modèle **non linéaire** : soit une transformation de features, soit un réseau à plusieurs couches (Jour 4).

Exercice 7

★★☆ Intermédiaire

a) Vérification prédiction par prédiction :

i	1	2	3	4	5	6	7	8	9	10
y_i	0	1	1	0	1	0	1	0	1	0
$\hat{y}_i$	0	1	0	0	1	1	1	0	0	0
Correct ?	✓	✓	×	✓	✓	×	✓	✓	×	✓

b) 7 corrects sur 10 : Accuracy $= \frac{7}{10} = \boxed{70\%}$.

c) Faux positifs (prédit 1, vrai 0) : $i = 6$ — **1 faux positif**.

Faux négatifs (prédit 0, vrai 1) : $i = 3$ et $i = 9$ — **2 faux négatifs**.

d) Prédire toujours 0 : les observations avec $y = 0$ sont $i = 1, 4, 6, 8, 10$ (5 sur 10). Accuracy naïve $= 5/10 = 50\%$. Notre classifieur (70%) fait bien mieux que le hasard.

À retenir

Un classifieur doit **au minimum** battre le classifieur naïf (toujours prédire la classe majoritaire). Pour un dataset équilibré 50/50, le seuil est 50%. Pour un dataset déséquilibré (ex. 90/10), il serait de 90%.

Exercice 8

★★☆ Intermédiaire

 CC BY-NC-SA 4.0

» Correction complète

```python
import numpy as np

data = np.array([
    [ 0,2,3, 150, 80,28.0],[ 1,1,2,  50,100,26.5],
    [ 1,3,4, 800, 40,35.0],[ 2,2,3, 200, 60,32.0],
    [ 2,4,4,3000, 20,38.0],[ 0,1,1,  30, 50,25.0],
    [ 3,3,3, 100, 80,34.5],[ 1,5,4,  15,100,42.0],
    [ 4,3,3, 500, 60,38.5],[ 5,4,4,1200, 40,44.0],
    [ 5,3,3, 300, 80,41.0],[ 6,5,4,2000, 30,48.0],
    [ 6,4,3, 150, 90,43.5],[ 7,5,4, 800, 50,50.0],
    [ 7,3,2,  80, 70,39.0],[ 4,6,5, 400,100,47.0],
    [ 8,4,4,1500, 40,51.0],[ 5,2,3,4000, 10,40.0],
    [ 9,5,4, 600, 60,52.0],[10,6,4,2500, 30,55.0],
    [10,4,3, 200, 80,48.5],[11,5,5,1000, 50,58.0],
    [12,7,4,3500, 20,60.0],[ 9,3,3,  60, 90,42.0],
    [13,6,4, 800, 70,62.0],[11,4,4,5000, 10,54.0],
    [15,8,4,1200, 50,65.0],[17,6,5,2000, 40,68.0],
    [20,7,4, 500, 60,72.0],[18,5,3, 100, 80,58.0]])

exp = data[:, 0]; sal = data[:, 5]
y = (sal > 45).astype(int)

# 1. P(salaire_eleve = 1)
p1 = np.mean(y)
print(f"P(sal>45k) = {p1:.2f}")  # 0.50

# 2. P(sal>45k | exp > 10)
mask_exp10 = exp > 10
p_cond = np.mean(y[mask_exp10])
print(f"P(sal>45k | exp>10) = {p_cond:.2f}")  # 1.00

# 3. P(sal>45k | nb_langages >= 5)
lang = data[:, 1]
mask_lang5 = lang >= 5
p_lang = np.mean(y[mask_lang5])
print(f"P(sal>45k | lang>=5) = {p_lang:.2f}")  # 0.77

# 4. Comparaison par seuils d'experience
print("\n--- Comparaison ---")
for seuil_exp in [3, 5, 8, 10]:
    mask = exp > seuil_exp
    if np.sum(mask) > 0:
        p = np.mean(y[mask])
        print(f"P(sal>45k | exp>{seuil_exp:2d}) = {p:.2f} "
              f"({np.sum(mask)} dev)")
```

Sortie attendue :

```
P(sal>45k) = 0.50
P(sal>45k | exp>10) = 1.00
P(sal>45k | lang>=5) = 0.77
```

 CC BY-NC-SA 4.0

```
--- Comparaison ---
P(sal>45k | exp> 3) = 0.62 (21 dev)
P(sal>45k | exp> 5) = 0.71 (17 dev)
P(sal>45k | exp> 8) = 0.92 (12 dev)
P(sal>45k | exp>10) = 1.00 (8 dev)
```

Plus le seuil d'expérience est élevé, plus la probabilité d'avoir un salaire élevé augmente.

⚠ Erreurs fréquentes

- Écrire `exp > 10` au lieu de `data[:, 0] > 10` — la variable `exp` doit être définie au préalable.
- Oublier de filtrer `y[mask]` — sans le masque, on calcule $P(\text{sal_eleve} = 1)$ sur tout le dataset.

Exercice 9

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

data = np.array([
    [ 0,2,28.0],[ 1,1,26.5],[ 1,3,35.0],[ 2,2,32.0],
    [ 2,4,38.0],[ 0,1,25.0],[ 3,3,34.5],[ 1,5,42.0],
    [ 4,3,38.5],[ 5,4,44.0],[ 5,3,41.0],[ 6,5,48.0],
    [ 6,4,43.5],[ 7,5,50.0],[ 7,3,39.0],[ 4,6,47.0],
    [ 8,4,51.0],[ 5,2,40.0],[ 9,5,52.0],[10,6,55.0],
    [10,4,48.5],[11,5,58.0],[12,7,60.0],[ 9,3,42.0],
    [13,6,62.0],[11,4,54.0],[15,8,65.0],[17,6,68.0],
    [20,7,72.0],[18,5,58.0]])

X = data[:, :2]
y = (data[:, 2] > 45).astype(int)

# Poids manuels
w = np.array([0.5, 0.8])
b = -4.0

# 1. Scores
scores = X @ w + b

# 2. Predictions
y_pred = (scores > 0).astype(int)

# 3. Accuracy
acc = np.mean(y_pred == y)
```

```python
print(f"Accuracy = {acc:.0%}")  # ~73%

# 4. Graphique
fig, ax = plt.subplots(figsize=(8, 6))
for c, color, label in [(0,'#e74c3c','<= 45k'),
                        (1,'#2ecc71','> 45k')]:
    mask = y == c
    ax.scatter(X[mask,0], X[mask,1], c=color,
               s=70, label=label, edgecolors='white')

x1_line = np.linspace(-1, 21, 100)
x2_line = -(w[0]/w[1]) * x1_line - b/w[1]
ax.plot(x1_line, x2_line, 'k--', lw=2,
        label='Frontiere')
ax.set_xlim(-1, 21); ax.set_ylim(0, 9)
ax.set_xlabel("Experience")
ax.set_ylabel("Nb langages")
ax.set_title(f"Frontiere (accuracy={acc:.0%})")
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

Sortie attendue : Accuracy $\approx 73\%$ + un graphique avec la frontière en pointillés.

Variante

Essayez d'autres poids pour améliorer l'accuracy. Par exemple $\mathbf{w} = (0{,}6,\ 0{,}5)$, $b = -4{,}5$. C'est exactement ce que le perceptron fait automatiquement (exercice 10).

Exercice 10

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

data = np.array([
    [ 0,2,28.0],[ 1,1,26.5],[ 1,3,35.0],[ 2,2,32.0],
    [ 2,4,38.0],[ 0,1,25.0],[ 3,3,34.5],[ 1,5,42.0],
    [ 4,3,38.5],[ 5,4,44.0],[ 5,3,41.0],[ 6,5,48.0],
    [ 6,4,43.5],[ 7,5,50.0],[ 7,3,39.0],[ 4,6,47.0],
    [ 8,4,51.0],[ 5,2,40.0],[ 9,5,52.0],[10,6,55.0],
    [10,4,48.5],[11,5,58.0],[12,7,60.0],[ 9,3,42.0],
    [13,6,62.0],[11,4,54.0],[15,8,65.0],[17,6,68.0],
    [20,7,72.0],[18,5,58.0]])

X = data[:, :2]
```

```python
y = (data[:, 2] > 45).astype(int)
n = len(y)

w = np.zeros(2)
b = 0.0
alpha = 0.1
n_epochs = 100

for epoch in range(n_epochs):
    n_errors = 0
    for i in range(n):
        z = w @ X[i] + b                  # score
        y_pred = 1 if z > 0 else 0        # prediction
        if y_pred != y[i]:                 # si erreur
            error = y[i] - y_pred          # +1 ou -1
            w = w + alpha * error * X[i]  # maj poids
            b = b + alpha * error          # maj biais
            n_errors += 1
    if epoch < 5 or epoch % 20 == 19:
        acc = np.mean([(1 if (w @ X[j] + b) > 0
                        else 0) == y[j]
                       for j in range(n)])
        print(f"Epoch {epoch+1:3d} : "
              f"erreurs={n_errors}, acc={acc:.0%}")

# Graphique
fig, ax = plt.subplots(figsize=(8, 6))
for c, color, label in [(0,'#e74c3c','<= 45k'),
                        (1,'#2ecc71','> 45k')]:
    mask = y == c
    ax.scatter(X[mask,0], X[mask,1], c=color,
               s=70, label=label, edgecolors='white')

if abs(w[1]) > 1e-6:
    x1 = np.linspace(-1, 21, 100)
    x2 = -(w[0]/w[1]) * x1 - b/w[1]
    ax.plot(x1, x2, 'k--', lw=2, label='Frontiere')

ax.set_xlim(-1, 21); ax.set_ylim(0, 9)
ax.set_xlabel("Experience")
ax.set_ylabel("Nb langages")
ax.set_title("Perceptron from scratch")
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

Sortie attendue :

```
Epoch   1 : erreurs=13, acc=57%
Epoch   2 : erreurs=7,  acc=80%
Epoch   3 : erreurs=5,  acc=87%
Epoch   4 : erreurs=4,  acc=87%
Epoch   5 : erreurs=4,  acc=87%
Epoch  20 : erreurs=3,  acc=90%
```

```
Epoch 100 : erreurs=3,  acc=90%
```

L'accuracy se stabilise autour de 87–90% après quelques époques. Les 3 erreurs restantes correspondent aux cas atypiques du dataset (junior bien payé, senior sous-payé).

⚠ Erreurs fréquentes

- Écrire `w += alpha * error * X[i]` sur un tableau NumPy modifie `w` **en place** — c'est correct mais attention aux effets de bord si on copie `w` ailleurs.
- Oublier `else 0` dans l'expression `1 if z > 0 else 0`.

Exercice 11

★★★ Avancé

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron as SkPerceptron

data = np.array([
    [ 0,2,28.0],[ 1,1,26.5],[ 1,3,35.0],[ 2,2,32.0],
    [ 2,4,38.0],[ 0,1,25.0],[ 3,3,34.5],[ 1,5,42.0],
    [ 4,3,38.5],[ 5,4,44.0],[ 5,3,41.0],[ 6,5,48.0],
    [ 6,4,43.5],[ 7,5,50.0],[ 7,3,39.0],[ 4,6,47.0],
    [ 8,4,51.0],[ 5,2,40.0],[ 9,5,52.0],[10,6,55.0],
    [10,4,48.5],[11,5,58.0],[12,7,60.0],[ 9,3,42.0],
    [13,6,62.0],[11,4,54.0],[15,8,65.0],[17,6,68.0],
    [20,7,72.0],[18,5,58.0]])

X = data[:, :2]
y = (data[:, 2] > 45).astype(int)
n = len(y)

# --- From scratch ---
w_fs = np.zeros(2); b_fs = 0.0; alpha = 0.1
hist_acc = []
for epoch in range(100):
    for i in range(n):
        z = w_fs @ X[i] + b_fs
        yp = 1 if z > 0 else 0
        if yp != y[i]:
            err = y[i] - yp
            w_fs += alpha * err * X[i]
            b_fs += alpha * err
    acc = np.mean([(1 if (w_fs@X[j]+b_fs)>0 else 0)
                   == y[j] for j in range(n)])
    hist_acc.append(acc)
acc_fs = hist_acc[-1]
```

 CC BY-NC-SA 4.0

```python

# --- scikit-learn ---
clf = SkPerceptron(max_iter=100, eta0=0.1,
                   random_state=42)
clf.fit(X, y)
acc_sk = clf.score(X, y)

# --- Tableau ---
print(f"{'Methode':<16}|{'w1':>8}|{'w2':>8}"
      f"|{'b':>8}|{'Acc':>7}")
print("-" * 52)
print(f"{'From scratch':<16}|{w_fs[0]:>8.3f}"
      f"|{w_fs[1]:>8.3f}|{b_fs:>8.3f}"
      f"|{acc_fs:>6.0%}")
print(f"{'sklearn':<16}|{clf.coef_[0][0]:>8.3f}"
      f"|{clf.coef_[0][1]:>8.3f}"
      f"|{clf.intercept_[0]:>8.3f}"
      f"|{acc_sk:>6.0%}")

# --- Graphiques ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

for c, color, label in [(0,'#e74c3c','<= 45k'),
                        (1,'#2ecc71','> 45k')]:
    mask = y == c
    ax1.scatter(X[mask,0], X[mask,1], c=color,
                s=60, label=label, edgecolors='white')

x1 = np.linspace(-1, 21, 100)
if abs(w_fs[1]) > 1e-6:
    ax1.plot(x1, -(w_fs[0]/w_fs[1])*x1 - b_fs/w_fs[1],
             'r-', lw=2, label=f'FS ({acc_fs:.0%})')
w_sk = clf.coef_[0]; b_sk = clf.intercept_[0]
if abs(w_sk[1]) > 1e-6:
    ax1.plot(x1, -(w_sk[0]/w_sk[1])*x1 - b_sk/w_sk[1],
             'g--', lw=2, label=f'sklearn ({acc_sk:.0%})')
ax1.set_xlim(-1,21); ax1.set_ylim(0,9)
ax1.set_xlabel("Experience")
ax1.set_ylabel("Nb langages")
ax1.legend(); ax1.grid(True, alpha=0.3)
ax1.set_title("Frontieres comparees")

ax2.plot(range(1, 101), hist_acc, 'b-', lw=1.5)
ax2.set_xlabel("Epoque")
ax2.set_ylabel("Accuracy")
ax2.set_title("Convergence from scratch")
ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()
```

Les deux frontières sont différentes car il existe plusieurs hyperplans séparateurs valides. Le perceptron converge vers **une** solution, pas nécessairement la même que sklearn (qui utilise une graine aléatoire différente).

CC BY-NC-SA 4.0

À retenir

Il n'y a pas de frontière "unique" pour un problème donné. Des algorithmes plus avancés (SVM, régression logistique) cherchent la "meilleure" frontière selon un critère précis.

Exercice 12

★★★ Avancé

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

# --- Partie 1 : echec sur XOR ---
X_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
y_xor = np.array([0, 1, 1, 0])

w = np.zeros(2); b = 0.0; alpha = 0.1
err_hist = []
for epoch in range(200):
    errs = 0
    for i in range(4):
        z = w @ X_xor[i] + b
        yp = 1 if z > 0 else 0
        if yp != y_xor[i]:
            e = y_xor[i] - yp
            w += alpha * e * X_xor[i]
            b += alpha * e
            errs += 1
    err_hist.append(errs)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
for c, col in [(0,'#e74c3c'),(1,'#2ecc71')]:
    mask = y_xor == c
    ax1.scatter(X_xor[mask,0], X_xor[mask,1],
                c=col, s=200, edgecolors='black',
                lw=2, zorder=3)
if abs(w[1]) > 1e-6:
    x1 = np.linspace(-0.5, 1.5, 100)
    ax1.plot(x1, -(w[0]/w[1])*x1 - b/w[1],
             'k--', lw=1.5, label='Derniere frontiere')
ax1.set_xlim(-0.5,1.5); ax1.set_ylim(-0.5,1.5)
ax1.set_title("XOR : impossible a separer")
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(err_hist, 'r-', lw=1)
ax2.set_xlabel("Epoque"); ax2.set_ylabel("Erreurs")
ax2.set_title("Oscillation (ne converge pas)")
ax2.grid(True, alpha=0.3)
```

 CC BY-NC-SA 4.0

```python
plt.tight_layout(); plt.show()

print("Le perceptron oscille : il ne converge JAMAIS.")

# --- Partie 2 (bonus) : avec feature x3 = x1*x2 ---
X_aug = np.column_stack([X_xor, X_xor[:,0]*X_xor[:,1]])
print(f"\nDataset augmente :\n{X_aug}")

w3 = np.zeros(3); b3 = 0.0
for epoch in range(100):
    errs = 0
    for i in range(4):
        z = w3 @ X_aug[i] + b3
        yp = 1 if z > 0 else 0
        if yp != y_xor[i]:
            e = y_xor[i] - yp
            w3 += alpha * e * X_aug[i]
            b3 += alpha * e
            errs += 1
    if errs == 0:
        print(f"\nConverge a l'epoque {epoch+1} !")
        break

acc = np.mean([(1 if (w3@X_aug[i]+b3)>0 else 0)
                == y_xor[i] for i in range(4)])
print(f"Accuracy = {acc:.0%}")
print(f"Poids = {w3}, biais = {b3:.1f}")
print("\nAvec la feature x1*x2, le probleme devient")
print("lineairement separable !")
```

Sortie attendue (partie 1) : Le graphique de droite montre les erreurs oscillant entre 1 et 2 sans jamais atteindre 0.

Sortie attendue (partie 2) :

```
Dataset augmente :
[[0 0 0]
 [0 1 0]
 [1 0 0]
 [1 1 1]]

Converge a l'epoque 7 !
Accuracy = 100%
```

La feature $x_3 = x_1 \times x_2$ rend le problème linéairement séparable dans $\mathbb{R}^3$.

Conclusion générale du Jour 3

Le perceptron est le premier neurone artificiel. Il ne résout que les problèmes **linéairement séparables**. Pour les problèmes non linéaires (comme le XOR), il faut soit ajouter des features manuellement, soit utiliser un **réseau de neurones multicouche** (Jour 4) qui crée automatiquement de nouvelles représentations dans ses couches cachées.

 CC BY-NC-SA 4.0

Transition vers le Jour 4

Le bonus de l'exercice 12 illustre l'idée clé des réseaux de neurones : les **couches cachées** jouent le rôle de "générateurs automatiques de features". Au lieu d'inventer manuellement $x_3 = x_1 x_2$, le réseau apprend lui-même les transformations nécessaires.

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0